

Hack the Gate

A ticket sale platform that keeps its promises

Project 1, concurrency. Four assignments, one specification.

1 The sale

In 2026 BTS will perform in Bogota. The stadium holds about 36,000 people. Somewhere between 150,000 and 300,000 tried to open the ticket page at the same second. Ticketing platforms like ticketmaster or tuboleta, in theory, follow the following promises/agreements:

- Nobody is sold a seat that does not exist.
- Nobody pays for a seat and then finds somebody else in it.
- The seat you were shown is the seat on your ticket.
- Two people click the same seat in the same millisecond and exactly one of them gets it.

You will build the back bones for such platforms, four times. The ticketing rules are written once and never change. What changes is the concurrency model underneath them:

1. Threads and Mutex
2. Software Transactional Memory (STM)
3. Communicating Sequential Processes (CSP)
4. Actors

The interesting question is not whether each one works. It is what each one costs, what each one makes easy, what each one makes possible (*e.g.*, queueing or the lack thereof) and what each one quietly makes impossible. By the fourth assignment you will have four answers to the same problem and you will be able to say which one you would put in front of a real sale, and why.

1.1 What is scaled down, and what is not

Nobody can run 200,000 real buyers on a laptop, and you will not be asked to. What is kept is the ratio, which is what makes the problem hard: the grader will send about 40,000 requests asking for roughly 72,000 seats at a venue that has 36,000. In this case, the overwhelming majority of requests must fail, and they must fail cleanly and quickly rather than hanging or lying. Two things a real platform has that are deliberately left out. There is no waiting room. There is no network, so the API is Elixir function calls and not HTTP.

2 The contract

The grader calls one module, `Gate.Venue`, which must declare `@behaviour Gate.API`. The behaviour is handed to you in `lib/gate/api.ex` and must not be edited. Nothing else in your code is ever called from outside.

A `ticket` is a map with the keys `:seat_id`, `:sector`, `:price_cop` and `:hold_id`. A `venue` is whatever your implementation needs; the grader only stores it and passes it back. A `hold_id` is opaque in the same way.

2.1 Seat identifiers

A seat id is "`<sector>:R<row>:S<seat>`", with the row and the seat padded to three digits and both counted from 1. Seat 34 of row 12 of the western lower ring is "`occidental_baja:R012:S034`", the same seat has the same name in every run and in every submission. `Gate.VenueSpec.seat_id/3` and `parse_seat_id/1` are provided.

2.2 The rules, one by one

There are no public tests. This section is therefore exact, and it is the whole contract. If you believe something here is ambiguous, ask, and the answer goes to everybody. The following rules describe the behaviors in `lib/gate/api.ex`:

Never blocks. `reserve/4` either serves the request now or fails. It does not queue, it does not wait for somebody else to cancel, and it does not retry on its own behalf. A caller that wants to try again does so itself.

All or nothing. A request for six seats that can only be half served fails and leaves the sector exactly as it was. Half a group is never held.

Splitting is allowed, sometimes. Mode `:any` may split a group across rows of the sector. Mode `:contiguous` may not, *i.e.*, all the seats come from one row and their numbers are consecutive.

Two distinct failure reasons. `:sold_out` means the sector does not have `qty` free seats at all. Whereas, `:no_contiguous_block` means it has them, but no run of `qty` consecutive free seats in any single row.

Hold ids are never reused. Not after the hold expires, is cancelled, or is confirmed. Not ever, within one venue.

Confirming twice fails. The first call to `confirm/2` on a live hold sells the seats. The second returns `{:error, :already_confirmed}`.

Cancelling twice is safe. Cancelling a hold with `cancel/2` that has already expired or already been cancelled returns `:ok`, because the caller asked for those seats to be given up and they are given up. Cancelling a confirmed hold returns `{:error, :already_confirmed}`, because a sale cannot be undone.

Unknown ids fail safely. A forged hold id, or the id of a hold that was cancelled, returns `:unknown_hold`. Your implementation must not crash on a hold id it never issued, whatever shape it has.

Availability is only a hint. `availability/2` is the number of free seats at the moment it is answered. It promises nothing about the next call, because other clients are running. It must not count held or sold seats.

Fairness is not required. There is no first come first served promise. What is required is that nobody is shut out completely: a client that keeps asking must eventually get somewhere while others are succeeding.

Snapshots are a consistent cut. Within one sector, the three seat sets that `snapshot/1` returns must describe one single instant of that sector's history. They are pairwise disjoint and together they name every seat of the sector. Across sectors the cut need not be a single global instant, because no seat ever moves between sectors, and freezing the whole venue at once is a different exercise. `:revenue` is the sum of the prices of the sold seats.

3 The venue

Provided in `lib/gate/venue_spec.ex` as for example `Gate.VenueSpec.el_campin/1`. Sector names and the three price tiers are the ones from the real sale. Row and seat counts are rounded to give the published capacity.

The smallest sector is the most expensive one, and the grader aims most of its traffic at the cheapest ones, which is what a real sale looks like. Your code must work for any spec of this shape and not only for this one.

4 Holds and expiry

A hold lives for `ttl_ms` milliseconds, taken from the venue spec:

- once `ttl_ms` has passed, the hold cannot be confirmed, and
- its seats are available to somebody else.

Because timing under load is not exact, the grader never tests the edge. A `confirm/2` issued well inside the lifetime must succeed. A `confirm` issued well after it must return `:expired`. In between, either answer is accepted. The strict test happens once the load has stopped and everything has had three lifetimes to expire, when there is no race left to lose: at that point no seat may still be held.

5 What you may not use

Construct	Why it is banned
<code>GenServer</code>	the actor model is what you are asked to build, not to call
<code>Agent</code>	a shared mutable cell is exactly the thing you have to implement
<code>Task</code>	spawning work and collecting it back is part of the exercise
<code>Supervisor</code> , <code>Registry</code>	project 2 territory
<code>GenStage</code> , <code>Flow</code>	concurrency libraries
<code>:ets</code> , <code>:dets</code>	a shared heap with atomic operations
<code>:mnesia</code>	may only be used to simulate the STM
<code>:persistent_term</code>	shared mutable memory outside your own processes
<code>:atomics</code> , <code>:counters</code>	hardware compare and swap, half a mutex for free
<code>:global</code> , <code>:pg</code>	a distributed lock manager, and process groups

What is left is what you need: `spawn`, `send`, `receive`, `Process`, pattern matching, and the standard data structures. Also banned is `Process.sleep` used as a way of waiting for something to happen. The absence of ETS is the interesting one. It means Elixir gives you no shared mutable memory at all. Any use of dependencies needs to be checked with staff first, though the answer is probably no.

6 Grading

Fully automated, out of 100. The grader is not published. The specification above is complete, so there is nothing in the grader that this document does not already tell you.

Category	Pts	What it asks
Functional	20	Does it follow the contract, one client at a time.
Safety	35	Under heavy load, is the database still consistent. No seat sold twice, no sale invented, no sale lost, every cut consistent, revenue matching the seats, and nothing still held once the sale is over.
Adversarial	15	Five load shapes chosen to break a design that is only accidentally correct: everybody on the smallest sector, groups fighting over the last seats, holds taken and given back with no pause, every hold abandoned so expiry does all the work, and a sector sold out to the very last seat.
Liveness	10	Did anything happen at all. The sale finishes inside its budget, every client completes its work, no client is shut out, and the only failures are the ones the contract allows.
Scalability	10	Does independent work run independently.
Performance	10	Throughput and tail latency against a floor for your model.

A banned construct means the submission is not graded. This is the same rule the course has always used for code that does not follow the specification.